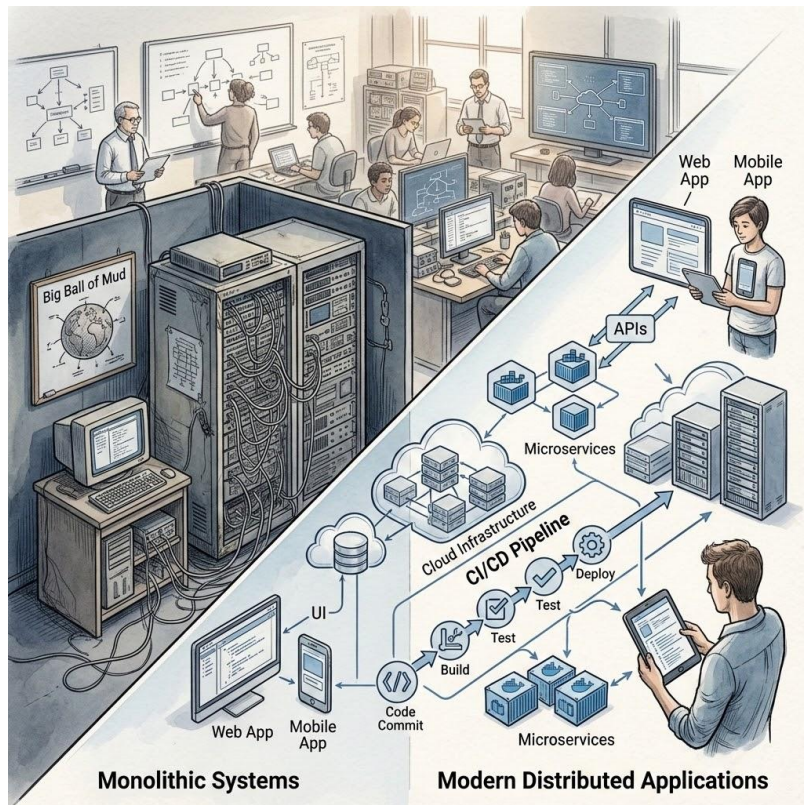


Segurança em Computação

Segurança de Aplicações Modernas e DevSecOps

Prof. Jean Everson Martina

O desenvolvimento de software mudou radicalmente



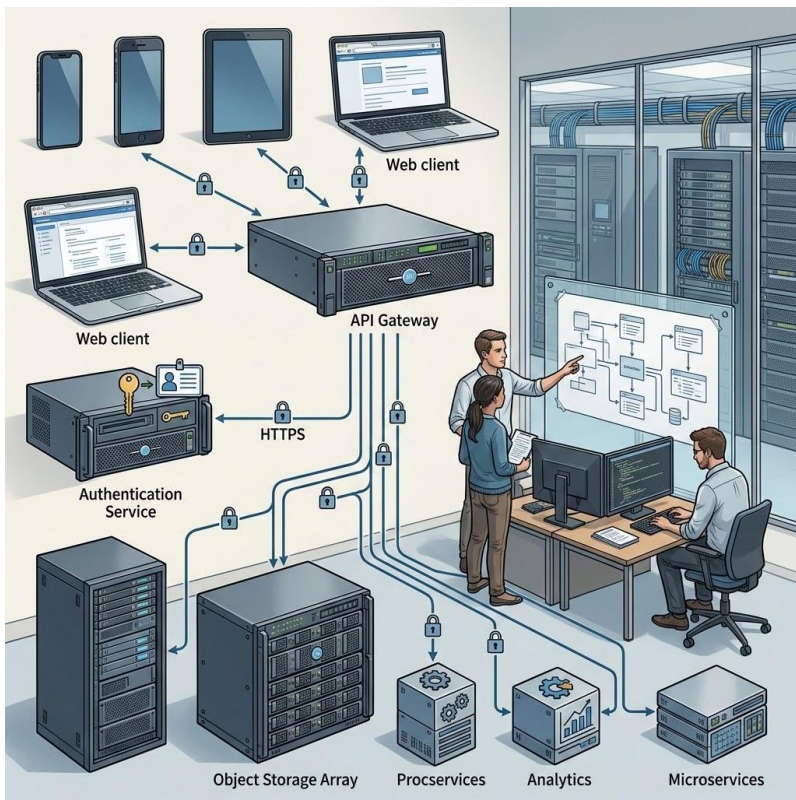
- Aplicações modernas deixaram de ser sistemas monolíticos isolados.
- Frontends web e mobile dependem continuamente de APIs e serviços distribuídos.
- Containers, cloud, CI/CD e automação passaram a fazer parte do software.
- O pipeline de build e deploy tornou-se infraestrutura crítica.
- Segurança deixou de ser auditoria final e passou a operar continuamente.
- O software moderno é um ecossistema distribuído, automatizado e permanentemente conectado.

Como aplicações modernas realmente funcionam

- Aplicações web e mobile normalmente operam como clientes de APIs.
- O backend centraliza autenticação, autorização e lógica crítica.
- Serviços modernos utilizam bancos, filas, cache e microsserviços.
- APIs HTTP/HTTPS tornaram-se a linguagem dominante da computação distribuída.
- Frontend e backend frequentemente evoluem de forma independente.
- Mesmo aplicações móveis normalmente são clientes HTTP sofisticados.



Aplicações modernas são ecossistemas HTTP/HTTPS



- APIs REST e GraphQL conectam aplicações, serviços e dispositivos.
- OAuth2, OpenID Connect e JWT tornaram-se mecanismos comuns de autenticação.
- Integrações entre serviços aumentam a superfície de ataque.
- Gateways de API concentram autenticação, rate limiting e observabilidade.
- Quebras de autorização em APIs estão entre os principais riscos atuais.
- Hoje a segurança de aplicações é inseparável da segurança de APIs.

Onde estão os riscos hoje

- Dependências vulneráveis podem comprometer aplicações inteiras.
- APIs expostas aumentam riscos de autorização e exposição de dados.
- Containers e cloud introduzem novos erros de configuração.
- Tokens, secrets e credenciais frequentemente vazam em pipelines.
- Ataques supply chain exploram bibliotecas e processos de build.
- Código gerado por IA pode introduzir vulnerabilidades silenciosas.
- Exemplos modernos
 - Log4Shell, MOVEit., SolarWinds, Dependências
- O atacante moderno frequentemente explora a cadeia de desenvolvimento.



Revisitando os fundamentos clássicos



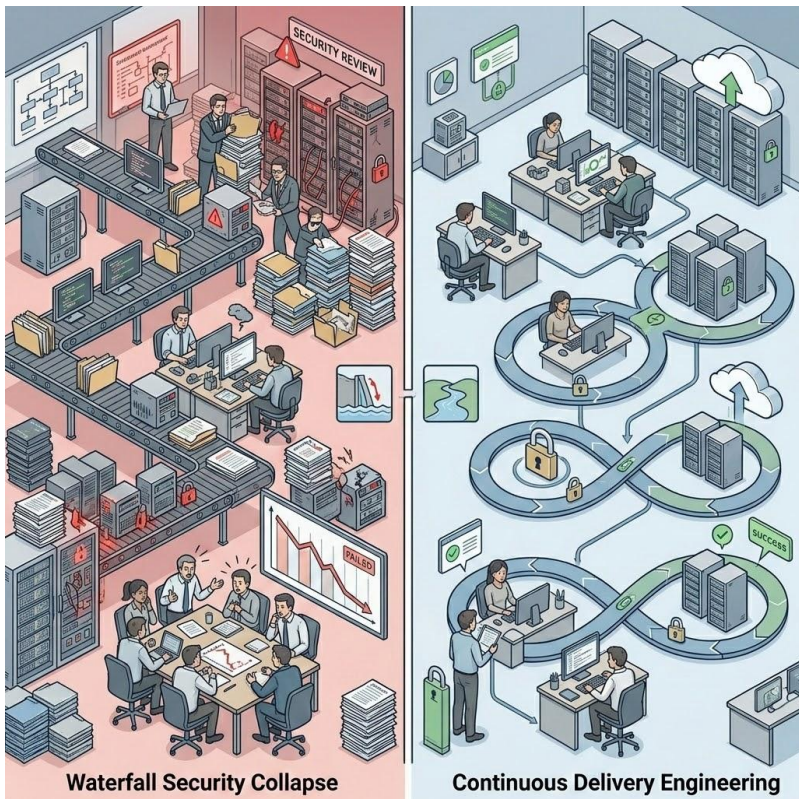
- Confidencialidade protege informações contra acesso indevido.
- Integridade garante consistência e não alteração indevida de dados.
- Disponibilidade mantém serviços operacionais e acessíveis.
- Autenticidade valida identidade e origem das operações.
- Não repúdio permite rastreabilidade e comprovação.
- Os princípios continuam válidos em ambientes distribuídos.
- Os fundamentos clássicos continuam corretos, o ambiente operacional mudou.

Observabilidade, resiliência e telemetria

- Sistemas modernos precisam produzir logs, métricas e traces continuamente.
- Observabilidade permite entender comportamento e falhas em runtime.
- Resiliência tornou-se requisito de segurança operacional.
- Segurança depende cada vez mais de telemetria e correlação de eventos.
- Sem visibilidade adequada, não existe detecção eficiente.
- Hoje segurança depende fortemente de observabilidade operacional.



O modelo antigo de segurança entrou em colapso



- Segurança tradicional acontecia apenas no final do desenvolvimento.
- Pentests e auditorias eram executados pouco antes do deploy.
- Correções tardias possuem alto custo operacional.
- Desenvolvimento e segurança operavam de forma isolada.
- Cloud e deploy contínuo tornaram esse modelo inviável.
- Não existe mais “janela final de segurança”.

O que é DevSecOps

- DevSecOps integra segurança ao ciclo completo de desenvolvimento.
- Segurança passa a ser contínua, automatizada e verificável.
- O pipeline executa validações técnicas automaticamente.
- Desenvolvimento, operações e segurança compartilham responsabilidades.
- Segurança deixa de ser apenas aprovação manual.
- DevSecOps é engenharia contínua de segurança.



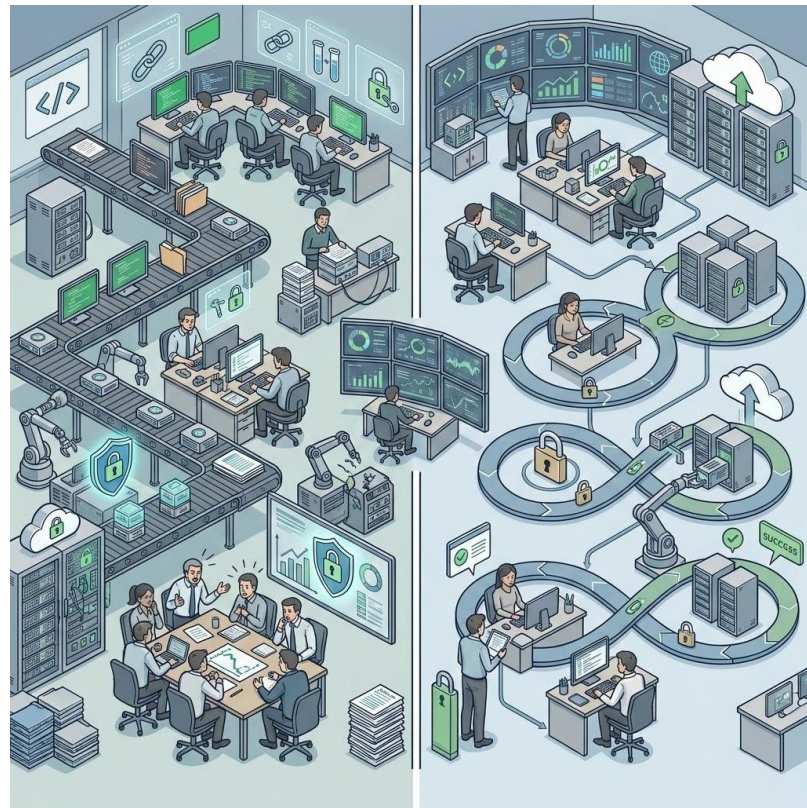
Shift Left, Shift Right e segurança contínua



- Shift Left antecipa validações de segurança para fases iniciais.
- Shift Right adiciona observabilidade e validação após deploy.
- Segurança moderna combina prevenção e monitoramento contínuo.
- Feedback rápido reduz custo e impacto de vulnerabilidades.
- O ciclo de segurança torna-se iterativo e permanente.
- Segurança moderna existe antes, durante e depois do deploy.

O pipeline moderno virou parte do sistema

- Pipelines automatizam build, testes, deploy e monitoramento.
- Credenciais e tokens frequentemente circulam no pipeline.
- GitHub Actions, GitLab CI e Jenkins executam código automaticamente.
- O pipeline possui acesso privilegiado à infraestrutura.
- Comprometer o pipeline pode comprometer toda a aplicação.
- O pipeline tornou-se infraestrutura crítica.



O pipeline também é superfície de ataque



- Secrets e tokens frequentemente vazam em automações.
- Dependências comprometidas podem ser distribuídas automaticamente.
- Build poisoning e supply chain attacks tornaram-se comuns.
- Actions e plugins externos ampliam riscos operacionais.
- O pipeline precisa de controles equivalentes aos sistemas de produção.
- Atacar o pipeline pode ser mais eficiente do que atacar a aplicação.

IA mudou a forma como software é produzido

- Ferramentas de IA aceleram geração de código e integração.
- IA frequentemente sugere snippets e bibliotecas inseguras.
- Desenvolvedores podem aceitar código sem compreender riscos.
- Aumento de produtividade não implica aumento de segurança.
- Segurança agora precisa validar código humano e código gerado automaticamente.
- IA aumenta produtividade, e também superfície de risco.



Prompt Injection e ferramentas assistidas por IA



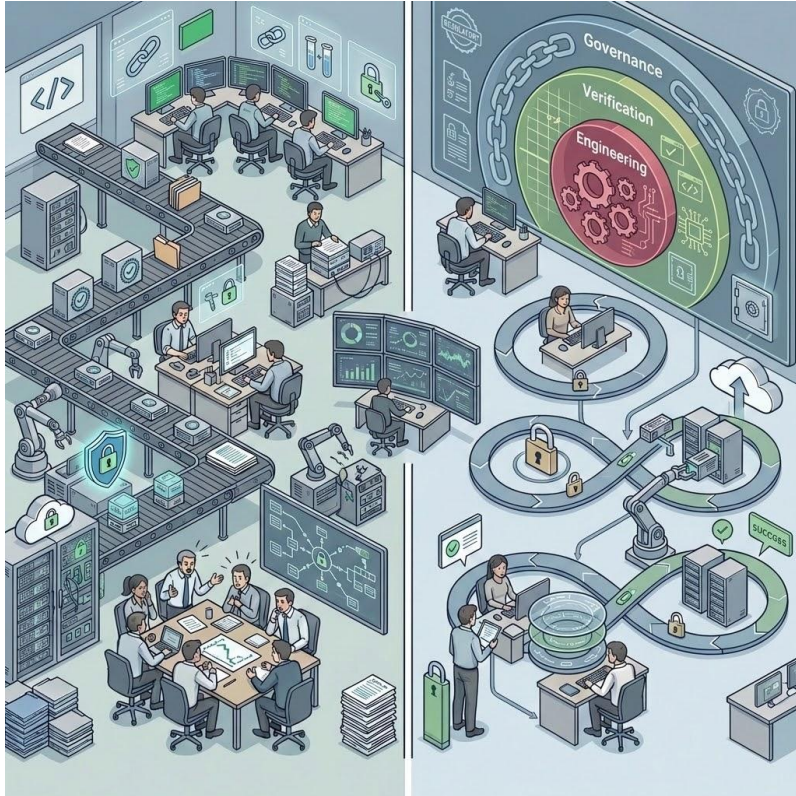
- Prompt Injection manipula comportamento de modelos de IA.
- Ferramentas de desenvolvimento assistidas também podem ser afetadas.
- Instruções ocultas podem influenciar geração de código inseguro.
- Plugins, extensões e documentação externa ampliam riscos.
- IA introduz novas superfícies de ataque dentro do desenvolvimento.
- A própria ferramenta de desenvolvimento tornou-se alvo.

Segurança contínua e CTEM

- CTEM significa Continuous Threat Exposure Management.
- O objetivo é medir continuamente exposição operacional.
- Vulnerabilidades precisam ser priorizadas conforme risco real.
- Segurança depende de observabilidade, métricas e telemetria.
- Segurança moderna funciona como ciclo contínuo de melhoria.
- Segurança moderna mede exposição continuamente.



Frameworks modernos de engenharia segura



- NIST SSDF define práticas modernas de desenvolvimento seguro.
- OWASP ASVS organiza requisitos técnicos de verificação.
- OWASP SAMM define maturidade organizacional em segurança.
- Secure by Design desloca responsabilidade para o produtor do software.
- Frameworks ajudam a transformar segurança em processo verificável.
- Segurança moderna é orientada por maturidade e processos estruturados.

Como tudo isso se conecta

- Frontend, APIs, backend e cloud operam como sistema integrado.
- O pipeline participa diretamente do ciclo operacional.
- Observabilidade conecta desenvolvimento, operação e resposta.
- Segurança moderna integra prevenção, validação e monitoramento.
- Vulnerabilidades podem surgir em qualquer camada do ecossistema.
- Aplicações modernas exigem visão sistêmica de engenharia e segurança.



O que construiremos nas próximas aulas



- Vulnerabilidades modernas em aplicações web, APIs e mobile.
- SAST, DAST, IAST, SCA e análise automatizada.
- SBOM e segurança de supply chain.
- Infrastructure as Code e Policy as Code.
- Red team, blue team e validação contínua.
- Integração entre desenvolvimento, operação e resposta.
- As próximas aulas transformarão teoria em engenharia prática.

Discussão final

- Segurança moderna é problema de desenvolvimento ou infraestrutura?
- Programadores deveriam entender pipelines e cloud?
- IA aumenta ou reduz segurança?
- É possível automatizar segurança sem gerar burocracia?
- O que acontece quando ninguém entende o fluxo completo?
- A segurança moderna depende de visão sistêmica e responsabilidade compartilhada.

